

# DIFFICULTY-CURVE CONDITIONED PROCEDURAL LEVEL GENERATION FOR GAMES USING DEEP GENERATIVE SEQUENCE MODELS

**AJITH K S**

*MCA S3*

*TCR25MCA-2008*

Mini Project

Guided By:

Sankaranarayanan P N

Department of Computer Applications  
Government Engineering College, Thrissur, Kerala





# Outline

1 Introduction

2 Literature Review

3 Existing Systems

4 Proposed System

5 Requirements Specification

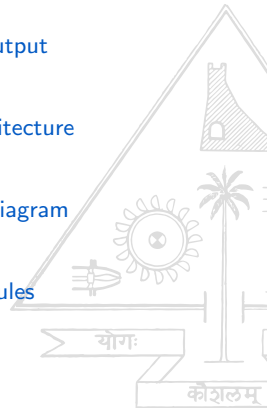
6 Input and Output

7 System Architecture

8 Data Flow Diagram

9 System Modules

10 References





# Introduction

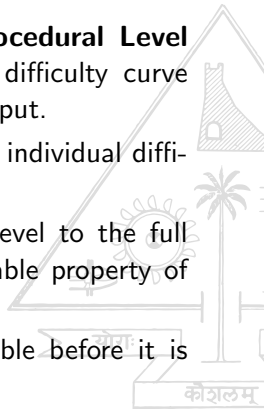
- Procedural Content Generation (PCG) is the algorithmic creation of game content – levels, terrain, items, rules – with limited or no direct human authorship.
- Level generation for platformers, dungeons, and maze-like environments is well studied because levels can be represented as sequences or grids of tiles.
- Dynamic Difficulty Adjustment (DDA) studies how a game can observe player performance and adapt challenge to keep players engaged.
- Most procedural level generators and difficulty-aware generators still treat each level as an independent artefact, conditioned on a single static difficulty label or a random seed.
- As a result, generators have no explicit notion of **pacing** – the deliberate rise, spike, and fall of challenge across a sequence of levels that designers use to sustain engagement.



# Introduction

## Objectives:

- Propose a **Difficulty-Curve-Conditioned Procedural Level Generator** that accepts a designer-authored difficulty curve (e.g., easy  $\rightarrow$  spike  $\rightarrow$  recovery  $\rightarrow$  ramp) as input.
- Generate a sequence of tile-based levels whose individual difficulties trace that curve point by point.
- Move difficulty conditioning from the single level to the full sequence, making pacing an explicit, controllable property of the generator.
- Guarantee that every generated level is solvable before it is surfaced to a designer or player.





# Literature Review

Weeks, M. and Davis, J., “Procedural Dungeon Generation for a 2D Top-down Game,” *Proc. 2022 ACM Southeast Conf. (ACMSE)*, 2022.

- Developed a procedural dungeon generator for 2D top-down environments, validated using search-based methods.
- Coupled randomized structural layouts with a localized variant of A\* search to navigate obstructions, locks, and keys.
- Extracted a structural complexity score from features such as path length, dead ends, room density, and puzzle dependencies.
- **Observation:** Treats generated dungeons as discrete, isolated instances rather than steps along an interconnected pacing arc.



# Literature Review

Shu, T., Liu, J., and Yannakakis, G. N., "Experience-Driven PCG via Reinforcement Learning: A Super Mario Bros Study," *Proc. IEEE Conf. on Games (CoG)*, 2021.

- Proposed EDRL: frames concatenation of level segments as a Markov Decision Process.
- Uses Proximal Policy Optimization (PPO) paired with GANs to dynamically assemble game sections.
- Reward landscape optimizes for structural diversity, historical novelty, and playability simultaneously.
- **Observation:** Optimized against static, generalized reward functions rather than a designer-authored trajectory of target difficulty values.



# Literature Review

González-Duque, M., Palm, R. B., and Risi, S., “Fast Game Content Adaptation Through Bayesian-based Player Modelling,” *Proc. IEEE Conf. on Games (CoG)*, 2021.

- Addresses the “cold-start” problem of profiling subjective player skill with minimal play sessions.
- Reframes player profiling as an optimization task solved via Gaussian Process Regression (FBCA).
- Converges on a personalized difficulty match within a minimal number of active interactions.
- **Observation:** Targets a single scalar value (e.g., completion time), not a full difficulty curve.



# Literature Review

González-Duque, M., Palm, R. B., Ha, D., and Risi, S., “Finding Game Levels with the Right Difficulty in a Few Trials through Intelligent Trial-and-Error,” *Proc. IEEE Conf. on Games (CoG)*, 2020, pp. 503–510.

- Proposes Intelligent Trial-and-Error (IT&E): bifurcates adaptation into offline and online stages.
- Offline: MAP-Elites builds a diverse behavioural archive of level variants mapped to simulated player profiles.
- Online: a Gaussian-Process-driven Bayesian optimizer locates a level matching a target win-rate within fewer than five trials.
- **Observation:** Operates on a trial-by-trial correction basis; does not sequence levels against a pre-determined macro difficulty curve.



# Literature Review

Gallotta, R., Todd, G., Zammit, M., Earle, S., Liapis, A., Togelius, J., and Yannakakis, G. N., "Large Language Models and Games: A Survey and Roadmap," *IEEE Transactions on Games*, 2024.

- Surveys the evolution of content generation from grammar-based systems to deep neural networks.
- Charts a shift toward hybrid architectures integrating LLMs, constraint satisfaction, and reinforcement learning.
- Notes that the frontier lies in steerable constraint frameworks letting designers specify exact experiential bounds, including difficulty curves.
- **Observation:** Identifies the need for such steerable control but provides no implementation.



# Literature Review

| Author                       | Approach                         | Brief Observation                                                                        |
|------------------------------|----------------------------------|------------------------------------------------------------------------------------------|
| Weeks & Davis (2022)         | Search-based (A*)                | Scores static structural complexity but evaluates levels as isolated instances.          |
| Shu et al. (2021)            | EDRL: PPO + GAN in an MDP        | Sequentially assembles levels toward a static, generalized reward, not a designer curve. |
| González-Duque et al. (2021) | Fast Bayesian Content Adaptation | Rapidly profiles player skill toward a single scalar target.                             |
| González-Duque et al. (2020) | Intelligent Trial-and-Error      | Matches a player within few trials; trial-by-trial, no macro arc.                        |
| Gallotta et al. (2024)       | Survey & roadmap                 | Identifies need for steerable difficulty control; no implementation.                     |



# Existing Systems

- Category 1 – generators conditioned on a single static difficulty label or random seed (n-gram statistical models, grammar-based, and search-based generators).
- Category 2 – dynamic difficulty adjustment systems, ranging from inventory-theoretic and fuzzy-logic interventions to RL-based sequence generators (e.g. Polymorph, EDRL) and Bayesian approaches (FBCA, IT&E).
- DDA systems adapt in response to observed player performance, either in real time or by iterative optimisation toward a single target value.
- None of these accept an explicit, multi-point difficulty curve authored by a designer in advance as their primary creative input.



# Limitations of Existing System

- Producing a sequence with designed pacing requires manually generating and re-ordering individual levels.
- DDA systems react level-by-level or encounter-by-encounter; they do not accept a whole-run difficulty curve before generation begins.
- Difficulty-aware generators (Polymorph, EDRL) optimise toward learned proxies, making it hard for a designer to specify precisely where a spike or recovery occurs.
- Standard PCG evaluation frameworks (expressive-range analysis, benchmarking testbeds) target single-level quality and diversity, not sequence pacing.
- Concatenating independently generated segments can break structures or produce unsolvable levels without a dedicated linking/repair mechanism.



# Proposed System

- A Difficulty-Curve-Conditioned Procedural Level Generator that treats the designer-authored difficulty trajectory as the primary creative object.
- Accepts a difficulty curve drawn across  $N$  levels (e.g. easy, spike, recovery, ramp) and outputs a sequence of tile-based levels tracing that curve.
- Combines a difficulty estimator, a trajectory-conditioned generative model, and an automated playability validator.
- A designer trajectory editor and interactive demo let the curve be drawn, the sequence generated, and the resulting pacing experienced directly.





# Advantages of Proposed System

- Pacing becomes an explicit, controllable input rather than an emergent or unspecified property.
- Difficulty estimation is grounded in measurable, interpretable proxies – path length, enemy density, required precision, branching complexity – calibrated against human-rated levels.
- Sequence-level conditioning closes the gap between per-level difficulty estimation and designed player-experience pacing.
- Automated playability validation guarantees every level in the sequence is solvable before it is shown.
- Reuses tractable PCG techniques – grid-based representation, tile-based generative modelling – avoiding 3D engine or physics complexity.



# Advantages of Proposed System

- The interactive demo lets a designer or evaluation panel play through a generated sequence and feel the intended pacing directly.

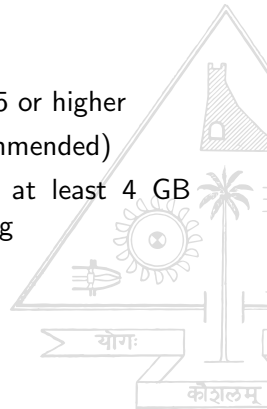




# Requirements Specification

## Hardware Requirements

- **Processor:** Intel® Core™ i5 / AMD Ryzen™ 5 or higher
- **Memory:** Minimum 8 GB RAM (16 GB recommended)
- **GPU (optional):** CUDA-capable GPU with at least 4 GB VRAM, to accelerate transformer/GAN training
- **Storage:** 20–50 GB free disk space





# Requirements Specification

## Software Development Requirements

- **Operating System:** Windows 10 / Ubuntu 20.04 LTS / macOS
- **Back End:** Python 3.10 or higher
- **IDE Used:** Visual Studio Code
- **Version Control:** Git
- **Development Tools and Frameworks:**
  - PyTorch
  - Pygame
  - Jupyter Notebook

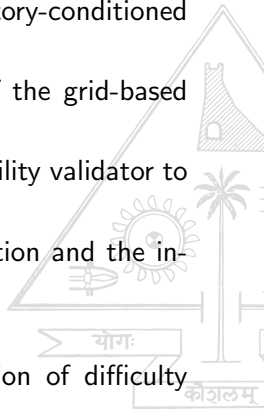




# Requirements Specification

## Key Python Libraries

- **PyTorch** – implements and trains the trajectory-conditioned generator (transformer/GAN).
- **NumPy** – efficient numerical manipulation of the grid-based tile representation.
- **A\* / pathfinding library** – used by the playability validator to confirm solvability.
- **Pygame** – renders generated levels for inspection and the interactive demo.
- **Flask** – serves the designer trajectory editor.
- **Matplotlib** – expressive-range-style visualisation of difficulty scores.





# Requirements Specification

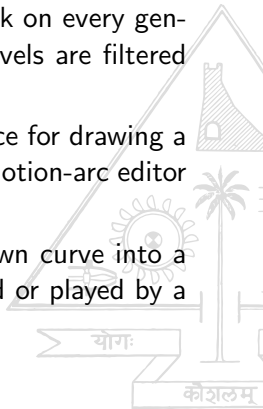
## Functional Requirements

- **Level Representation Setup:** A shared grid-based 2D tile format used consistently by the estimator, generator, and validator.
- **Difficulty Estimator:** Scores a level using measurable proxies – path length, enemy density, required precision, branching complexity – combining heuristics with a learned component calibrated on human-rated levels.
- **Trajectory-Conditioned Generator:** An autoregressive transformer or a GAN with a sequence-linking loop, conditioned on a target difficulty vector mapped across each slot in the sequence.



# Requirements Specification

- **Playability Validator:** Runs an A\*-based check on every generated level before it is surfaced; unsolvable levels are filtered out or regenerated.
- **Designer Trajectory Editor:** A simple interface for drawing a difficulty curve across  $N$  levels, similar to an emotion-arc editor applied to game pacing.
- **Interactive Demo Interface:** Converts a drawn curve into a generated run of levels that can be auto-played or played by a tester/evaluation panel.





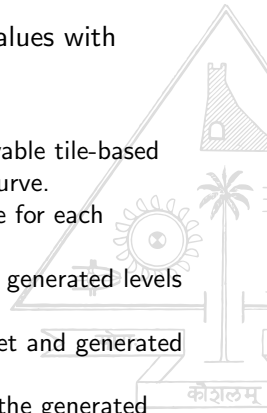
# Requirements Specification

## Non-Functional Requirements

- **Scalability:** Handles varying sequence lengths without disproportionate generation time or loss of curve consistency.
- **Usability:** The trajectory editor requires no specialised knowledge of the generative model or difficulty scoring.
- **Reliability:** Consistently produces playable levels and handles malformed or extreme curve inputs gracefully.
- **Maintainability:** Loosely coupled modules – representation, estimator, generator, validator, interface – so any one can be swapped independently.
- **Performance:** Generation and validation complete quickly enough to support an interactive, demonstration-friendly workflow.



- **Input:** Designer-defined difficulty curve and level generation parameters.
- **Input Format:** Sequence of target difficulty values with user-defined generation settings.
- **Output:**
  - **Generated Level Sequence:** A series of playable tile-based game levels following the specified difficulty curve.
  - **Difficulty Analysis:** Estimated difficulty score for each generated level.
  - **Playability Report:** Validation confirming all generated levels are solvable.
  - **Difficulty Curve Graph:** Comparison of target and generated difficulty progression.
  - **Interactive Demo:** Playable visualization of the generated level sequence.





# System Architecture

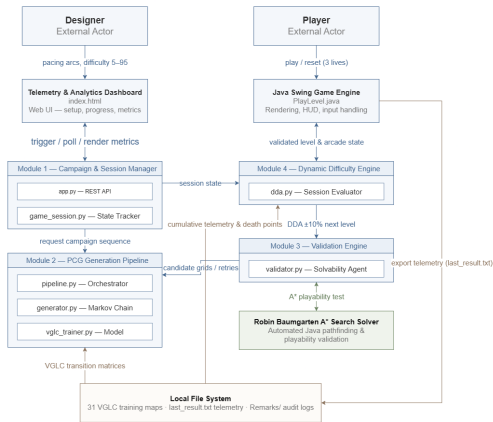


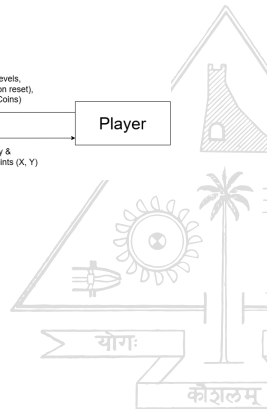
Figure 1: Super Mario Bros PCG & DDA Framework Architecture



# Data Flow Diagrams



Figure 2: Level 0 DFD





# Data Flow Diagrams

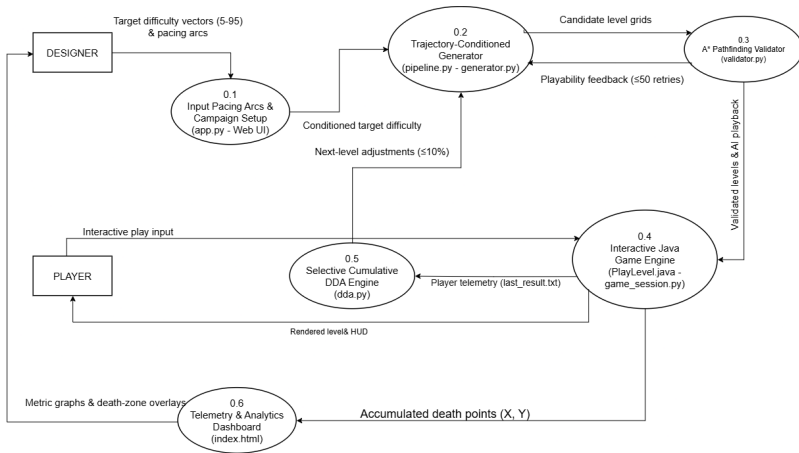


Figure 3: Level 1 DFD



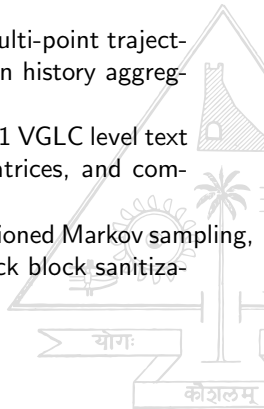
# System Modules

- **Web Engine & Telemetry Interface Module:** Harmonizes metrics, designer curve inputs, and telemetry onto a single unified 0 to 100% scale while rendering cumulative death zone telemetry via a modern glassmorphism interface.
- **Cumulative Session DDA Engine Module:** Dynamically adjusts level difficulty based on cumulative player telemetry across all preceding levels played in the active session using exponential decay weighting.
- **Trajectory-Conditioned Sequence Generator Module:** Coordinates multi-point trajectory generation and transitions Mario PCG into a holistic multi-level campaign engine through three sub-components:



# System Modules

- *Pipeline* (*pipeline.py*) — coordinates full multi-point trajectory generation, DDA simulation, and campaign history aggregation.
- *VGLC Trainer* (*vglc\_trainer.py*) — parses 31 VGLC level text files, extracts 2nd-order column transition matrices, and computes difficulty metrics.
- *Generator* (*generator.py*) — executes conditioned Markov sampling, post-processing decoration enhancements, stack block sanitization, and level generation.





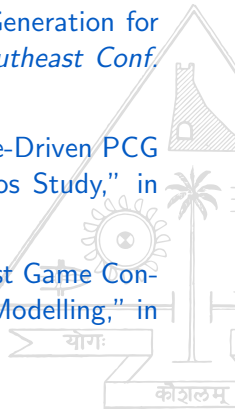
# System Modules

- **A\* Pathfinding Playability Validator Module:** Evaluates candidate level grids using an automated Robin Baumgarten search agent to test if Mario can navigate from start to finish within 200 seconds while tracking jump heights, gap clearances, and hazard avoidance.
- **Game Session Manager:** Manages player arcade campaign state, life system, coin accumulation, and session progression tracking, passing lives smoothly into the Java MarioWorld HUD.
- **Java Swing Entry Point (PlayLevel.java):** Executes human interactive play and AI playback, returning full telemetry via `last_result.txt` to inform future DDA adjustments.



# References

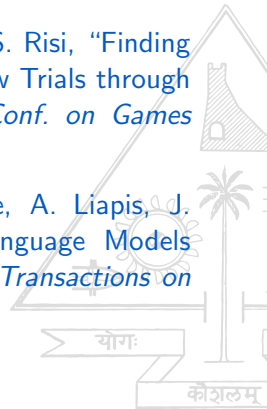
- [1] M. Weeks and J. Davis, “Procedural Dungeon Generation for a 2D Top-down Game,” in *Proc. 2022 ACM Southeast Conf. (ACMSE)*, Virtual Event, USA, 2022, pp. 60–66.
- [2] T. Shu, J. Liu and G. N. Yannakakis, “Experience-Driven PCG via Reinforcement Learning: A Super Mario Bros Study,” in *Proc. IEEE Conf. on Games (CoG)*, 2021.
- [3] M. González-Duque, R. B. Palm and S. Risi, “Fast Game Content Adaptation Through Bayesian-based Player Modelling,” in *Proc. IEEE Conf. on Games (CoG)*, 2021.





# References

- [4] M. González-Duque, R. B. Palm, D. Ha and S. Risi, “Finding Game Levels with the Right Difficulty in a Few Trials through Intelligent Trial-and-Error,” in *Proc. IEEE Conf. on Games (CoG)*, 2020, pp. 503–510.
- [5] R. Gallotta, G. Todd, M. Zammit, S. Earle, A. Liapis, J. Togelius and G. N. Yannakakis, “Large Language Models and Games: A Survey and Roadmap,” *IEEE Transactions on Games*, 2024.





Thank You

